

Carreras de caracoles — documento de respuesta

Aplicación: <https://caracoles-staging.mangobinario.com>

Catálogo de componentes: <https://caracoles-staging.mangobinario.com/componentes>

Repositorio: <https://github.com/yosa/luis-2036>

Autor: Luis Heredia

Resumen en cifras

Qué	Resultado
Requisitos funcionales	24 de 24 terminados, cada uno con evidencia enlazada en el repositorio
Mínimos de validez	Los 4 cumplidos y probados de punta a punta
Pruebas automatizadas	137 unitarias y de integración (40 API, 97 frontend), 6 E2E y 98 aserciones de Postman
Adicionales	Los 2 terminados: aplicación desplegada y propuesta de base de datos
Decisiones documentadas	8 registros de decisión (ADR), todos aceptados
Tiempo de trabajo	Cerca de 2 h 55 min sobre el alcance

1. Resumen del proceso

- 1. Análisis.** Reescribí el enunciado como requisitos con identificador (RF, RNF, entregables y adicionales). Llevé una matriz de estado en la que cada requisito solo se marca como terminado si tiene evidencia enlazada: una prueba o el archivo que lo implementa.
- 2. Estándares antes que código.** React y Express no son mi stack habitual (trabajo con Laravel, Vue/Quasar y Nuxt). Antes de programar escribí mis estándares para React y Express siguiendo el patrón de los que ya uso. Cada decisión relevante quedó en un registro de decisiones (ADR) con sus alternativas.
- 3. Contrato primero.** Definí el contrato de SnailPay como un esquema compartido entre el API y el frontend, e implementé el API con sus escenarios y pruebas.
- 4. Frontend por ramas pequeñas.** Base técnica, tema, cliente HTTP, componentes, autenticación, dashboard y recarga. Cada rama incluyó sus pruebas y la actualización de la documentación, y la revisé en el navegador antes del merge.
- 5. Cierre.** Pruebas E2E sobre el build de producción, colección de Postman, despliegue en AWS y publicación del repositorio.

2. Decisiones principales

- **Monorepo con npm workspaces** (api, frontend y shared). El contrato de SnailPay es un solo esquema zod que usan los dos lados, así que no puede divergir. Elegí npm y no pnpm para que ejecutarlo no requiera instalar nada más.
- **SnailPay responde siempre con su contrato de proveedor**, incluso ante JSON malformado, rate limit o un error inesperado. Así el cliente tiene un único parser y los nueve campos obligatorios están en toda respuesta.
- **Escenarios elegidos por los datos de la tarjeta**, como en los entornos de prueba de las pasarelas reales:
 - rechazos (fondos insuficientes, tarjeta bloqueada, vencida, CVV o fecha incorrectos, tarjeta desconocida);
 - error del sistema, por tarjeta o por variable de entorno;
 - timeout real: el servidor tarda 12 s y el cliente corta a los 8 s.

La combinación de éxito se evalúa antes que el vencimiento, para que siga aprobándose después de diciembre de 2026.

- **Regla contra falsos éxitos.** El saldo solo aumenta si se cumplen cinco condiciones: HTTP 201, estado aprobado, código de autorización, monto igual al solicitado y un identificador de cobro no aplicado antes. Una única operación del monedero puede subir el saldo, y toda respuesta se valida contra el contrato antes de usarse.
- **Contraseña:** hash PBKDF2-SHA256 con 600 000 iteraciones (mínimo de OWASP), sal aleatoria por usuario y comparación en tiempo constante, todo con la Web Crypto API. La contraseña nunca se guarda en claro. Un correo inexistente y una contraseña incorrecta dan el mismo mensaje y tardan lo mismo (se verifica contra un

hash señuelo). Documenté el límite: hacerlo en el navegador protege una copia del almacenamiento, no a quien controla el navegador.

- **LocalStorage detrás de un acceso tipado y versionado:** cada clave tiene esquema, se valida al leer y, si algo está corrupto, vuelve al valor por defecto sin romper la aplicación.
- **Número de tarjeta y CVV:** se incluyen en la respuesta y se guardan porque lo pide el alcance. Aun así, la interfaz los muestra enmascarados, el CVV no vuelve a mostrarse y los logs del API los redactan. Dejé escrito cómo sería en producción: tokenización y sin almacenar el CVV.
- **Gráficas con datos simulados deterministas** por usuario y día: 6 carreras entre 6 caracoles. La suma de victorias siempre es 6 y cada apuesta ganada coincide con el ganador de su carrera.
- **Estado con Zustand**, por su parecido con los stores de Pinia que uso a diario.

3. Herramientas, librerías y plantillas

- **Stack:** React 19, Vite 8, TypeScript 6, React Router 8, Zustand, zod, Recharts, SASS con CSS Modules; Express 5, pino, helmet, cors, express-rate-limit; Vitest, Testing Library, supertest, Cypress y Newman.
- **Versiónes:** fijé TypeScript 6 y ESLint 9 porque las herramientas de lint todavía no soportan las versiones más recientes.
- **Sin plantilla ni librería de componentes.** Construí a mano los componentes (botón, campo, alerta, tarjetas y gráficas) sobre tokens de diseño propios.
- **Diseño:** la paleta y la tipografía (Fredoka y Nunito) partieron de las recomendaciones del plugin ui-ux-pro-max. Las ajusté hasta que todos los pares de color cumplieran contraste WCAG AA en modo claro y oscuro. Los colores de las gráficas los validé con el script de una guía de visualización de datos: el gris que había elegido para “perdidas” no pasaba el croma mínimo y lo cambié por violeta.
- **Íconos:** lucide.

4. Uso de inteligencia artificial y forma de validación

- **Claude Code** (modelo Claude Opus 5.5) apoyó todo el proceso: análisis del enunciado, redacción de estándares y documentación, generación de código y pruebas, y depuración.
- **Claude in Chrome** (extensión conectada a Claude Code) fue el banco de pruebas exploratorio. Con él recorrí cada pantalla en un navegador real y revisé la consola, el almacenamiento y el diseño en móvil.
- **Proceso:** trabajé en ramas pequeñas. Antes de cada merge revisaba el resumen del diff, las pruebas y el resultado en el navegador, y decidía si se integraba o se ajustaba.
- **Qué detecté yo en esas revisiones:**
 - que la base del frontend iba demasiado de golpe, así que pedí dividirla en ramas con pruebas;
 - que los componentes no tenían dónde validarse, lo que llevó a un catálogo de componentes, que después pedí publicar;
 - que la documentación no se actualizaba al avanzar;
 - un tooltip tapado por el texto central de la dona;
 - que el script de despliegue exponía datos de mi infraestructura en un repositorio que sería público. Se hizo genérico y se reescribió el historial antes de publicar.

También cuestioné la necesidad del hash en el frontend; lo revisamos contra su registro de decisión y se mantuvo.

- **Validación:**
 - todas las pruebas en verde antes de cada merge;
 - “canarios” (errores metidos a propósito) para comprobar que el type-check, el lint y el E2E sí fallan;
 - recorridos reales contra el API local y contra la versión desplegada.

La bitácora completa está en el repositorio (docs/07-entrega/uso-de-ia.md).

5. Pruebas implementadas y razón de su elección

Probé primero lo que, si falla, hace daño sin que se note a simple vista: acreditar saldo cuando no se debía, guardar la contraseña de forma débil, que SnailPay responda distinto de lo documentado, perder la sesión o el saldo al recargar, y gráficas incoherentes con sus reglas.

- **API (40 pruebas).**
 - Cada escenario de SnailPay con su estado, detalle y HTTP.
 - Los campos obligatorios en toda respuesta, incluidos 400, 422, 429 y 503.
 - La caída simulada, que nunca aprueba.
 - Que los logs nunca contengan la tarjeta ni el titular.
 - CORS restringido y cabeceras de seguridad.
- **Frontend (97 pruebas).**
 - La regla contra falsos éxitos, condición por condición, incluido un “aprobado” con otro monto.
 - El hash de la contraseña.
 - Almacenamiento corrupto o bloqueado.
 - Los cuatro mínimos de validez con el router real.
 - La recarga con cada resultado (aprobada, rechazada, error del sistema, timeout, sin red).
 - Las invariantes del día simulado sobre 200 semillas.
- **E2E con Cypress (6 pruebas)** sobre el build de producción: registro, cierre de sesión, inicio de sesión, persistencia al recargar, guard de rutas y recarga aprobada, rechazada y con timeout real.
- **Smoke de Postman con Newman:** 13 solicitudes y 98 aserciones contra el API real.

Los selectores de prueba usan label, rol o texto visible, como un usuario. Eso obliga a que la interfaz sea accesible.

6. Funcionalidades terminadas

- **Registro e inicio de sesión:**
 - registro con validaciones;
 - inicio y cierre de sesión;
 - persistencia al recargar;
 - dashboard accesible solo con sesión, que regresa a la ruta pedida tras iniciar sesión.
- **Dashboard:** nombre del usuario, saldo inicial de \$0, dona de apuestas y barras de victorias por caracol. Cada gráfica tiene resumen en texto, tooltip y tabla alternativa.
- **SnailPay:** cobro exitoso, ocho errores de transacción, error del sistema documentado, timeout, rate limit y los campos obligatorios en toda respuesta.
- **Recarga:**
 - el saldo se actualiza al instante y se persiste;
 - cada resultado tiene su mensaje;
 - historial con la tarjeta enmascarada;
 - panel con las tarjetas de prueba.
- **Adicionales 1 y 2 terminados:** aplicación desplegada y propuesta de base de datos.

7. Funcionalidades incompletas o problemas conocidos

No quedan funcionalidades del alcance sin terminar ni problemas abiertos. Estas son limitaciones asumidas y documentadas:

- La autenticación y el saldo viven en el navegador, como pide la simulación local, así que quien edite LocalStorage puede alterarlos. La versión de producción es la propuesta de base de datos.
- No hay deduplicación de cobros en el servidor (Idempotency-Key), porque SnailPay no guarda estado. El cliente evita acreditar dos veces el mismo cobro.
- En la versión desplegada la caída por variable de entorno está apagada: el error del sistema se prueba con la tarjeta que termina en 0503.
- El despliegue es manual, con un script, y sin CI.
- Solo verifiqué la interfaz en Chrome, en escritorio y en móvil emulado.

8. Tiempo aproximado invertido

Cerca de 2 h 55 min de trabajo sobre el alcance, contados con las marcas de tiempo de commits y archivos, sin las esperas entre revisiones. Aparte, unos 17 minutos en escribir mis estándares para React y Express, que me servirán para otros proyectos. El detalle por actividad está en docs/07-entrega/registro-de-tiempo.md.

9. Repositorio público

<https://github.com/yosa/luis-2036>. Las instrucciones para ejecutar el frontend y el API, correr las pruebas y reproducir cada respuesta de SnailPay están en el README y en la carpeta docs.

Adicional 1: aplicación desplegada

- **URL:** <https://caracoles-staging.mangobinario.com>. Se revisa sin credenciales: basta con registrarse, y las tarjetas de prueba aparecen en la pantalla de recarga.
- **Catálogo de componentes:** <https://caracoles-staging.mangobinario.com/componentes> muestra el sistema visual y los estados de cada componente, sin iniciar sesión.
- **Plataforma:** AWS.
 - **API:** en Lambda (Node 22) detrás de API Gateway, con dominio propio: <https://api-caracoles-staging-us-east-1.mangobinario.com>.
 - **Sitio:** en S3 y CloudFront.
- **Implementación:**
 - El API es la misma aplicación Express que corre en local, envuelta con serverless-http y empaquetada en un solo archivo.
 - El sitio se construye con la URL del API y se sube a S3. Una función de CloudFront resuelve cada ruta de la aplicación a su página principal, así que recargar cualquier pantalla funciona.
 - El API solo acepta peticiones del sitio publicado (CORS) y el sitio no se indexa en buscadores.
 - Un script reproduce el despliegue completo y verifica el resultado; los datos de la infraestructura no están en el repositorio.
- **Limitaciones:**
 - el primer cobro tras un rato de inactividad tarda un poco más (arranque en frío);
 - los datos de cada evaluador viven en su propio navegador;
 - el costo en reposo es cero.

Adicional 2: propuesta de base de datos

- **Tecnología:** PostgreSQL serverless, con Kysely como query builder tipado y migraciones versionadas. Encaja con Lambda sin agotar conexiones.
- **Qué se almacena y cómo se relaciona:**
 - Usuarios, con el hash de la contraseña (Argon2id) calculado en el servidor.
 - Sesiones, identificadas por una cookie HttpOnly.
 - Monederos, uno por usuario. El saldo es la suma de un libro de movimientos inmutable.
 - Cobros de SnailPay: sin número completo de tarjeta ni CVV, solo los últimos cuatro dígitos.
 - Caracoles, carreras (con su ganador) y apuestas (usuario, carrera, caracol y resultado).
- **Restricciones:** el saldo no puede ser negativo, y cada cobro y cada llave de idempotencia son únicos.
- **Cambios necesarios:**
 - El backend incorpora autenticación, monedero y carreras, y llama a SnailPay por su cuenta.
 - La recarga se acredita en una sola transacción (cobro, movimiento y saldo) con bloqueo de fila, así que un reintento no puede acreditar dos veces.
 - El frontend deja de leer LocalStorage y consulta el API, la tarjeta se tokeniza con el SDK de la pasarela y LocalStorage queda solo para preferencias.